

# 1000 Python Practice Questions

From Beginner to Advanced

10 parts • 49 topics • 1000 questions

# How to Use This Book

- Questions are grouped into 10 parts of 100, ordered by difficulty. Work through them in order; each part builds on the previous one.
- **Solve first, look up later.** Write your own solution before searching. Run it, test it with edge cases (empty input, zero, negative numbers, very large values).
- **Tick as you go.** Use the box next to each question to mark it done. Revisit the ones that took you more than 30 minutes.
- **Reuse ideas.** Many later questions can be solved by combining earlier ones. Try solving the same problem in two ways (loop vs. recursion, loop vs. comprehension).
- **Suggested pace:** 10 questions a day finishes the set in about 100 days. Beginners may spend longer on Parts 1 to 3, which is fine.
- **Study first.** Each topic starts with a green box listing the concepts to learn. Study those, try small examples, then solve the questions below the box.
- Questions are numbered 1 to 1000 so you can track progress and discuss specific problems with others.

## Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| Part 1: Python Basics (Beginner)                                  | Q1-100    |
| Part 2: Conditionals and Loops (Beginner)                         | Q101-200  |
| Part 3: Strings (Beginner to Intermediate)                        | Q201-300  |
| Part 4: Lists, Tuples and Arrays (Intermediate)                   | Q301-400  |
| Part 5: Dictionaries and Sets (Intermediate)                      | Q401-500  |
| Part 6: Functions and Recursion (Intermediate)                    | Q501-600  |
| Part 7: Files, Exceptions and Modules (Intermediate)              | Q601-700  |
| Part 8: Object-Oriented Programming (Intermediate to Advanced)    | Q701-800  |
| Part 9: Intermediate to Advanced Python Features                  | Q801-900  |
| Part 10: Advanced Algorithms, Concurrency and Projects (Advanced) | Q901-1000 |

# Part 1: Python Basics (Beginner)

Questions 1 to 100

## Output and Variables

### STUDY THESE TOPICS FIRST

- print() function, sep and end arguments
- Variables and assignment (single, multiple, chained)
- Variable naming rules, keywords and conventions (snake\_case)
- Comments (# and docstrings)
- Escape characters (\n, \t, \\, quotes) and raw strings
- Triple-quoted strings
- f-strings, str.format() and % formatting
- type() and id()

- ☐ 1. Print "Hello, World!" to the screen.
- ☐ 2. Print your name and age on two separate lines using one print() call.
- ☐ 3. Print the numbers 1 to 5 on one line separated by dashes using the sep argument.
- ☐ 4. Print three words on one line without a newline between separate print() calls using end.
- ☐ 5. Create a variable for your city and print it.
- ☐ 6. Swap the values of two variables a and b without using a third variable.
- ☐ 7. Assign the same value 10 to three variables in a single statement.
- ☐ 8. Assign three different values to three variables in one line.
- ☐ 9. Print the type of the values 5, 5.0, "5", True and None.
- ☐ 10. Use an f-string to print "My name is X and I am Y years old".
- ☐ 11. Use str.format() to print the same sentence as the previous question.
- ☐ 12. Use the % operator to format a float to 2 decimal places.
- ☐ 13. Print a tab-separated table of three rows and two columns using escape characters.
- ☐ 14. Print a string that contains both single and double quotes.
- ☐ 15. Print a multi-line message using triple quotes.
- ☐ 16. Print a backslash and a Windows file path correctly using raw strings.
- ☐ 17. Check which of these are valid variable names: 2name, \_name, my-name, myName, class.
- ☐ 18. Write a program that prints the id() of a variable before and after reassigning it.
- ☐ 19. Use the keyword module to print the list of all Python keywords.
- ☐ 20. Add single-line and multi-line comments to explain a small program.

## Data Types and Type Conversion

### STUDY THESE TOPICS FIRST

- Built-in types: int, float, str, bool, complex, None
- Type casting: int(), float(), str(), bool()
- Truthy and falsy values
- bin(), oct(), hex(), int(x, base)
- ord() and chr()
- Mutable vs. immutable types
- Floating point precision, round() and the decimal module

- ☐ 21. Convert the string "123" to an integer and add 7 to it.
- ☐ 22. Convert the integer 45 to a string and concatenate it with "years".
- ☐ 23. Convert the float 9.99 to an integer and explain what happens to the decimals.
- ☐ 24. Convert the string "3.14" to a float.
- ☐ 25. Convert the number 0 and the number 5 to booleans and print them.
- ☐ 26. Print the boolean value of "", " ", [], [0] and None.
- ☐ 27. Convert an integer to its binary, octal and hexadecimal string forms.
- ☐ 28. Convert the binary string "1010" to a decimal integer.
- ☐ 29. Convert a character to its ASCII code and an ASCII code back to a character.
- ☐ 30. Check the type of the result of 10 / 2 and 10 // 2.
- ☐ 31. Demonstrate that int, float and str are immutable by trying to change a string character.
- ☐ 32. Find the largest integer Python can handle by computing 2 \*\* 1000.
- ☐ 33. Show floating point imprecision with 0.1 + 0.2 and fix the comparison using round().
- ☐ 34. Use the decimal module to add 0.1 and 0.2 exactly.
- ☐ 35. Create a complex number and print its real and imaginary parts.

## Operators

### STUDY THESE TOPICS FIRST

- Arithmetic operators: + - \* / // % \*\*
- Operator precedence and parentheses
- Comparison operators and chained comparison
- Logical operators: and, or, not
- Assignment and augmented assignment operators
- Identity (is) vs. equality (==)
- Bitwise operators and shifts
- Walrus operator :=

- ☐ 36. Add, subtract, multiply and divide two numbers entered in code and print each result.
- ☐ 37. Find the quotient and remainder of 47 divided by 5.
- ☐ 38. Compute 2 raised to the power 10 using the \*\* operator.
- ☐ 39. Find the square root of 144 without the math module.
- ☐ 40. Compute the average of three numbers.

- ☐ 41. Check if a number is even or odd using the modulus operator.
- ☐ 42. Convert 250 minutes into hours and minutes.
- ☐ 43. Convert a temperature from Celsius to Fahrenheit.
- ☐ 44. Convert kilometers to miles.
- ☐ 45. Calculate simple interest given principal, rate and time.
- ☐ 46. Calculate compound interest for 5 years.
- ☐ 47. Calculate the area and perimeter of a rectangle.
- ☐ 48. Calculate the area and circumference of a circle with radius 7.
- ☐ 49. Calculate the area of a triangle using base and height.
- ☐ 50. Calculate the area of a triangle using Heron's formula.
- ☐ 51. Calculate the hypotenuse given two sides using the Pythagorean theorem.
- ☐ 52. Use augmented assignment operators (`+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`) on one variable and print after each.
- ☐ 53. Demonstrate the difference between `==` and `is` with two lists.
- ☐ 54. Use chained comparison to check if  $10 < x < 20$ .
- ☐ 55. Demonstrate `and`, `or` and `not` with a truth table for two variables.
- ☐ 56. Use bitwise AND, OR, XOR, NOT and shifts on two integers.
- ☐ 57. Explain with code why `5 // -2` gives `-3`.
- ☐ 58. Use the walrus operator `:=` inside an `if` statement.
- ☐ 59. Compute the BMI of a person given weight and height.
- ☐ 60. Calculate the total cost of items including 18 percent GST.

## User Input

### STUDY THESE TOPICS FIRST

- `input()` always returns a string
- Converting input with `int()` and `float()`
- Reading multiple values with `split()` and `map()`
- `strip()` and basic input cleaning
- Formatting output with f-strings and `round()`
- Extracting digits using `//` and `%`

- ☐ 61. Ask the user for their name and greet them.
- ☐ 62. Read two integers from the user and print their sum.
- ☐ 63. Read a float from the user and print its square.
- ☐ 64. Read a full sentence and print it in upper case.
- ☐ 65. Read three numbers in one line separated by spaces and print their average.
- ☐ 66. Read a comma-separated line of numbers and print their total.
- ☐ 67. Ask for a user's birth year and print their age.
- ☐ 68. Ask for a radius and print the area of the circle rounded to 2 decimals.
- ☐ 69. Read a character and print its ASCII value.
- ☐ 70. Read a number and print its multiplication table up to 10 using a loop.
- ☐ 71. Ask for first and last names and print them as "Last, First".

- ☐ 72. Read length and width and print the area and perimeter.
- ☐ 73. Read an integer and print its last digit.
- ☐ 74. Read a 3-digit integer and print the sum of its digits.
- ☐ 75. Read a 4-digit integer and print it reversed.
- ☐ 76. Read the hours worked and hourly rate and print the salary.
- ☐ 77. Read marks of five subjects and print total, average and percentage.
- ☐ 78. Read the price and quantity of an item and print the bill.
- ☐ 79. Read an amount in rupees and print how many 500, 100, 50 and 10 notes are needed.
- ☐ 80. Read seconds and print them as hh:mm:ss.

## Math and Built-in Functions

### STUDY THESE TOPICS FIRST

- `abs()`, `round()`, `min()`, `max()`, `sum()`, `pow()`, `divmod()`
- The math module: `sqrt`, `ceil`, `floor`, `factorial`, `gcd`, `pi`, `log`, `trig` functions
- The random module: `randint`, `choice`, `shuffle`, `random`, `seed`
- The datetime module: `date`, `time`, `datetime`, `timedelta`, `strftime`
- `time.sleep()` and `time.time()`

- ☐ 81. Use `abs()`, `round()`, `min()`, `max()` and `sum()` on a small list of numbers.
- ☐ 82. Use the math module to print `ceil` and `floor` of 4.3 and -4.3.
- ☐ 83. Use `math.pi` and `math.e` to print their values to 4 decimals.
- ☐ 84. Use `math.factorial` to compute 10 factorial.
- ☐ 85. Use `math.gcd` to find the greatest common divisor of 48 and 18.
- ☐ 86. Use `math.sqrt` and `math.pow` to compute a few values.
- ☐ 87. Use `math.sin`, `cos` and `tan` for 30, 45 and 60 degrees.
- ☐ 88. Use `math.log`, `log10` and `log2` on suitable numbers.
- ☐ 89. Use `divmod()` to get quotient and remainder together.
- ☐ 90. Use `pow()` with three arguments for modular exponentiation.
- ☐ 91. Generate a random integer between 1 and 100 using the random module.
- ☐ 92. Pick a random item from a list of five names.
- ☐ 93. Shuffle a list of numbers and print it.
- ☐ 94. Generate a random float between 0 and 1.
- ☐ 95. Set a seed and show that random numbers repeat.
- ☐ 96. Print the current date and time using `datetime`.
- ☐ 97. Print today's date in the format DD-MM-YYYY.
- ☐ 98. Calculate the number of days between two given dates.
- ☐ 99. Print the day of the week for a given date.
- ☐ 100. Use `time.sleep()` to pause a program for two seconds and print before and after.

# Part 2: Conditionals and Loops (Beginner)

Questions 101 to 200

## if / elif / else

### STUDY THESE TOPICS FIRST

- Boolean expressions and comparison operators
- if, elif and else syntax and indentation
- Nested conditions
- Combining conditions with and, or, not
- Ternary (conditional) expression
- match-case statement (Python 3.10+)
- Truthiness checks in conditions
- Modulus for divisibility tests

- ☐ **101.** Check whether a number is positive, negative or zero.
- ☐ **102.** Check whether a number is even or odd.
- ☐ **103.** Find the largest of two numbers.
- ☐ **104.** Find the largest of three numbers without using max().
- ☐ **105.** Find the smallest of three numbers.
- ☐ **106.** Check whether a year is a leap year.
- ☐ **107.** Check whether a person is eligible to vote based on age.
- ☐ **108.** Assign a grade (A to F) based on a marks percentage.
- ☐ **109.** Check whether a character is a vowel or a consonant.
- ☐ **110.** Check whether a character is an uppercase letter, lowercase letter, digit or special symbol.
- ☐ **111.** Check whether a number is divisible by both 3 and 5.
- ☐ **112.** Print the name of the day for a number between 1 and 7.
- ☐ **113.** Print the name of the month for a number between 1 and 12.
- ☐ **114.** Print the number of days in a given month and year.
- ☐ **115.** Check whether three sides can form a triangle.
- ☐ **116.** Classify a triangle as equilateral, isosceles or scalene.
- ☐ **117.** Check whether three angles form a valid triangle.
- ☐ **118.** Build a simple calculator that takes two numbers and an operator.
- ☐ **119.** Calculate electricity bill using slab rates for units consumed.
- ☐ **120.** Calculate income tax using simple slabs.
- ☐ **121.** Check whether a given point lies inside a circle.
- ☐ **122.** Check if a given time in hours falls in morning, afternoon, evening or night.
- ☐ **123.** Use the ternary conditional expression to find the larger of two numbers.
- ☐ **124.** Use match-case to print the name of a weekday from its number.
- ☐ **125.** Check whether a number is within a given range using chained comparison.

## for Loops

### STUDY THESE TOPICS FIRST

- for loop syntax and iterating over sequences
- range(start, stop, step)
- enumerate() and iterating strings and lists
- Accumulator and counter patterns
- Digit extraction with % and //
- Nested loops (introduction)
- Loop variables and scope

- ☐ **126.** Print numbers from 1 to 10.
- ☐ **127.** Print numbers from 10 down to 1.
- ☐ **128.** Print all even numbers from 1 to 50.
- ☐ **129.** Print the first 10 multiples of 7.
- ☐ **130.** Print the sum of the first N natural numbers.
- ☐ **131.** Print the sum of the squares of the first N natural numbers.
- ☐ **132.** Print the factorial of a number using a loop.
- ☐ **133.** Print the multiplication table of a given number.
- ☐ **134.** Print the multiplication tables from 1 to 10 in a grid.
- ☐ **135.** Print all numbers between 1 and 100 divisible by 7.
- ☐ **136.** Count how many digits are in a number using a loop.
- ☐ **137.** Calculate the sum of the digits of a number.
- ☐ **138.** Reverse the digits of a number.
- ☐ **139.** Check whether a number is a palindrome.
- ☐ **140.** Compute the power  $a^b$  using a loop only.
- ☐ **141.** Print the characters of a string one per line.
- ☐ **142.** Print each character of a string with its index using enumerate.
- ☐ **143.** Count the vowels in a string.
- ☐ **144.** Print all numbers from 1 to 100 that are divisible by 3 or 5.
- ☐ **145.** Print the sum of all odd numbers from 1 to 99.
- ☐ **146.** Print the average of numbers from 1 to N.
- ☐ **147.** Find the sum of the series  $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{N}$ .
- ☐ **148.** Find the sum of the series  $1 - \frac{1}{2} + \frac{1}{3} - \dots$  up to N terms.
- ☐ **149.** Print the ASCII table for uppercase letters.
- ☐ **150.** Loop over a list of names and print a greeting for each.

## while Loops

### STUDY THESE TOPICS FIRST

- while loop syntax and loop conditions
- Avoiding infinite loops
- Sentinel values and input validation loops
- Counter-controlled vs. condition-controlled loops
- The Euclidean algorithm
- Loop with else
- Simple menu-driven loops
- Digit processing with % and //

- ☐ **151.** Print numbers from 1 to 10 using a while loop.
- ☐ **152.** Keep asking for a number until the user enters a positive value.
- ☐ **153.** Sum numbers entered by the user until they enter 0.
- ☐ **154.** Count down from 10 to 1 and print "Liftoff!".
- ☐ **155.** Find the factorial of a number using a while loop.
- ☐ **156.** Find the GCD of two numbers using the Euclidean algorithm.
- ☐ **157.** Find the LCM of two numbers.
- ☐ **158.** Find how many times a number can be divided by 2 before it becomes odd.
- ☐ **159.** Convert a decimal number to binary using a while loop.
- ☐ **160.** Convert a binary number to decimal without int(base).
- ☐ **161.** Check whether a number is an Armstrong number.
- ☐ **162.** Print all Armstrong numbers between 1 and 1000.
- ☐ **163.** Check whether a number is a perfect number.
- ☐ **164.** Print all perfect numbers below 10000.
- ☐ **165.** Check whether a number is a strong number (sum of digit factorials).
- ☐ **166.** Generate the Collatz sequence for a given number.
- ☐ **167.** Simulate a number guessing game with hints "too high" and "too low".
- ☐ **168.** Implement a password retry system that allows only 3 attempts.
- ☐ **169.** Calculate how many years it takes for an investment to double.
- ☐ **170.** Find the smallest number whose square is greater than 1000.
- ☐ **171.** Print the digits of a number from right to left.
- ☐ **172.** Find the sum of digits repeatedly until a single digit remains.
- ☐ **173.** Count the number of trailing zeros in a factorial.
- ☐ **174.** Simulate an ATM menu loop with balance, deposit, withdraw and exit.
- ☐ **175.** Use a while-else construct to report when a search fails.

## break, continue, pass and Nested Loops

### STUDY THESE TOPICS FIRST

- break, continue and pass statements
- for-else and while-else
- Nested loop execution order
- Prime number checking logic
- Fibonacci logic with two variables
- Pattern printing: rows, columns, spaces and print(end="")
- Row and column index relationships

- ☐ **176.** Print numbers 1 to 20 but stop when you hit a multiple of 13.
- ☐ **177.** Print numbers 1 to 20 skipping multiples of 3 using continue.
- ☐ **178.** Use pass in a placeholder if block.
- ☐ **179.** Use for-else to determine if a number is prime.
- ☐ **180.** Print all prime numbers between 1 and 100.
- ☐ **181.** Print the first N prime numbers.
- ☐ **182.** Check whether a number is a prime using the square root optimisation.
- ☐ **183.** Find the prime factors of a number.
- ☐ **184.** Print the Fibonacci series up to N terms.
- ☐ **185.** Print the Fibonacci numbers below 1000.
- ☐ **186.** Check whether a number is part of the Fibonacci sequence.
- ☐ **187.** Print twin primes below 200.
- ☐ **188.** Print a right-angled triangle pattern of stars.
- ☐ **189.** Print an inverted right-angled triangle pattern of stars.
- ☐ **190.** Print a pyramid pattern of stars.
- ☐ **191.** Print a diamond pattern of stars.
- ☐ **192.** Print a hollow square pattern.
- ☐ **193.** Print a number triangle: 1, 12, 123, 1234 and so on.
- ☐ **194.** Print Floyd's triangle with N rows.
- ☐ **195.** Print Pascal's triangle with N rows.
- ☐ **196.** Print the alphabet triangle A, AB, ABC, ...
- ☐ **197.** Print a checkerboard pattern using 0 and 1.
- ☐ **198.** Find all pairs (a, b) between 1 and 20 where  $a + b = 20$ .
- ☐ **199.** Find all Pythagorean triples with sides below 50.
- ☐ **200.** Print the multiplication table 1 to 10 formatted in aligned columns.

# Part 3: Strings (Beginner to Intermediate)

Questions 201 to 300

## String Basics

### STUDY THESE TOPICS FIRST

- Strings are immutable sequences
- Indexing (positive and negative) and len()
- Concatenation and repetition
- Case methods: upper, lower, title, capitalize, swapcase
- strip, lstrip, rstrip
- count, find, rfind, index, replace
- startswith, endswith
- Checks: isdigit, isalpha, isalnum, isupper, islower
- zfill, ljust, rjust

- ☐ **201.** Find the length of a string without using len().
- ☐ **202.** Print the first and last character of a string.
- ☐ **203.** Print a string in reverse using slicing.
- ☐ **204.** Print every second character of a string.
- ☐ **205.** Print the middle character(s) of a string.
- ☐ **206.** Concatenate two strings and repeat the result three times.
- ☐ **207.** Convert a string to upper, lower, title and swapcase.
- ☐ **208.** Capitalize only the first letter of a sentence.
- ☐ **209.** Remove leading and trailing whitespace from a string.
- ☐ **210.** Remove only leading or only trailing characters from a string.
- ☐ **211.** Count the occurrences of a character in a string.
- ☐ **212.** Count the occurrences of a substring in a string.
- ☐ **213.** Find the first and last index of a substring.
- ☐ **214.** Replace all spaces in a string with underscores.
- ☐ **215.** Replace only the first occurrence of a word in a sentence.
- ☐ **216.** Check if a string starts with and ends with given substrings.
- ☐ **217.** Check whether a string contains only digits, only letters or only alphanumerics.
- ☐ **218.** Check whether a string is all uppercase or all lowercase.
- ☐ **219.** Compare two strings ignoring case.
- ☐ **220.** Pad a string with zeros to a width of 8 on the left.

## String Slicing and Methods

### STUDY THESE TOPICS FIRST

- Slicing syntax [start:stop:step] and negative steps
- split, join, partition, splitlines
- center, ljust, rjust
- translate and maketrans
- Building strings with loops and join
- Immutability: creating new strings instead of editing
- String comparison

- ☐ **221.** Extract the substring between two given indexes.
- ☐ **222.** Remove the first and last characters of a string.
- ☐ **223.** Remove the character at a given index from a string.
- ☐ **224.** Insert a character at a given index in a string.
- ☐ **225.** Split a sentence into words and print each word.
- ☐ **226.** Join a list of words into a sentence with a hyphen separator.
- ☐ **227.** Split a string on commas and strip each item.
- ☐ **228.** Split a multi-line string into lines.
- ☐ **229.** Partition a string at the first "=" into key and value.
- ☐ **230.** Center a string in a field of width 30 using a fill character.
- ☐ **231.** Left-justify and right-justify a string in a field of width 20.
- ☐ **232.** Use translate and maketrans to remove all digits from a string.
- ☐ **233.** Use str.zfill to format numbers as 001, 002, 003.
- ☐ **234.** Slice a string into chunks of size 3.
- ☐ **235.** Rotate a string left by k positions.
- ☐ **236.** Rotate a string right by k positions.
- ☐ **237.** Check whether one string is a rotation of another.
- ☐ **238.** Print a string's characters in alternating upper and lower case.
- ☐ **239.** Swap the case of every letter without using swapcase().

## String Algorithms

### STUDY THESE TOPICS FIRST

- Iterating over characters
- Counting with dictionaries and Counter
- Two-pointer technique
- Sorting with sorted() and key functions
- Sets for uniqueness
- Palindrome and anagram logic
- Word splitting and joining
- Character frequency tables

- ☐ **240.** Check whether a string is a palindrome.

- ☐ **241.** Check whether a sentence is a palindrome ignoring spaces, punctuation and case.
- ☐ **242.** Count vowels, consonants, digits and spaces in a string.
- ☐ **243.** Count the number of words in a sentence.
- ☐ **244.** Find the longest word in a sentence.
- ☐ **245.** Find the shortest word in a sentence.
- ☐ **246.** Reverse the order of words in a sentence.
- ☐ **247.** Reverse each word in a sentence but keep the word order.
- ☐ **248.** Capitalize the first letter of every word without using title().
- ☐ **249.** Find the frequency of each character in a string.
- ☐ **250.** Find the first non-repeating character in a string.
- ☐ **251.** Find the first repeating character in a string.
- ☐ **252.** Find the most frequent character in a string.
- ☐ **253.** Remove all duplicate characters from a string while preserving order.
- ☐ **254.** Check whether two strings are anagrams.
- ☐ **255.** Group a list of words into anagram groups.
- ☐ **256.** Check whether a string contains all letters of the alphabet (pangram).
- ☐ **257.** Remove all vowels from a string.
- ☐ **258.** Remove all punctuation from a string.
- ☐ **259.** Find all unique words in a sentence ignoring case.
- ☐ **260.** Sort the characters of a string alphabetically.
- ☐ **261.** Sort the words of a sentence alphabetically.
- ☐ **262.** Sort the words of a sentence by length.
- ☐ **263.** Find the longest common prefix among a list of strings.
- ☐ **264.** Find the longest common substring of two strings.

## Encoding, Formatting and Conversions

### STUDY THESE TOPICS FIRST

- ord() and chr() with modular arithmetic (ciphers)
- Dictionaries for lookup tables (Morse, Roman numerals)
- Run-length encoding logic
- Format specifiers: width, precision, commas, percentage, signs
- str.encode() and bytes.decode()
- The base64 module
- Case conversion logic

- ☐ **265.** Implement the Caesar cipher to encrypt a message.
- ☐ **266.** Decrypt a message encrypted with the Caesar cipher.
- ☐ **267.** Implement the ROT13 cipher.
- ☐ **268.** Encode a string in Morse code.
- ☐ **269.** Encode a string using run-length encoding.
- ☐ **270.** Decode a run-length encoded string.

- ☐ **271.** Convert a string of digits to its English words (123 to "one two three").
- ☐ **272.** Convert a number to its Roman numeral.
- ☐ **273.** Convert a Roman numeral to an integer.
- ☐ **274.** Convert a snake\_case string to camelCase.
- ☐ **275.** Convert a camelCase string to snake\_case.
- ☐ **276.** Convert a string to a URL slug.
- ☐ **277.** Mask all but the last 4 digits of a card number.
- ☐ **278.** Format a number with thousand separators and two decimals.
- ☐ **279.** Format a number as a percentage with one decimal place.
- ☐ **280.** Convert an integer to a string with a leading plus or minus sign.
- ☐ **281.** Encode a string to bytes using UTF-8 and decode it back.
- ☐ **282.** Encode and decode a string with base64.

## Validation and Text Processing

### STUDY THESE TOPICS FIRST

- String methods for validation (isdigit, isalpha, etc.)
- Stack idea for bracket matching
- Splitting text into sentences and words
- Counter and most\_common()
- any() and all()
- The textwrap module
- Dynamic programming idea for edit distance (preview)

- ☐ **283.** Validate whether a string is a valid integer without using try/except.
- ☐ **284.** Validate a password: length at least 8, with upper, lower, digit and special character.
- ☐ **285.** Validate an email address using string methods only.
- ☐ **286.** Validate a phone number of 10 digits.
- ☐ **287.** Validate an IPv4 address string.
- ☐ **288.** Check whether brackets in a string are balanced.
- ☐ **289.** Check whether a string is a valid identifier name.
- ☐ **290.** Count the number of sentences in a paragraph.
- ☐ **291.** Find the most common word in a paragraph.
- ☐ **292.** Print a word frequency table of a paragraph sorted by count.
- ☐ **293.** Replace every digit in a string with its square.
- ☐ **294.** Extract all digits from a string and sum them.
- ☐ **295.** Extract all numbers (including multi-digit) from a string and sum them.
- ☐ **296.** Extract all capital letters from a string.
- ☐ **297.** Remove extra spaces between words so only single spaces remain.
- ☐ **298.** Wrap a long paragraph to lines of at most 40 characters using textwrap.
- ☐ **299.** Compute the Hamming distance of two equal-length strings.
- ☐ **300.** Compute the Levenshtein (edit) distance of two strings.

# Part 4: Lists, Tuples and Arrays (Intermediate)

Questions 301 to 400

## List Basics

### STUDY THESE TOPICS FIRST

- Creating lists and indexing
- append, insert, extend, remove, pop, clear
- index, count, sort, reverse
- sorted() vs. list.sort() and the key argument
- Slicing and slice assignment
- Copying: assignment, copy(), slicing, deepcopy
- The in operator and len()
- str.join() with lists

- ☐ **301.** Create a list of five fruits and print the third item.
- ☐ **302.** Print the last item of a list using negative indexing.
- ☐ **303.** Add an item to the end, the start and the middle of a list.
- ☐ **304.** Remove an item by value, by index and the last item.
- ☐ **305.** Extend one list with another and compare with append.
- ☐ **306.** Find the length of a list, and the index of a given element.
- ☐ **307.** Check whether an element exists in a list.
- ☐ **308.** Count how many times an element occurs in a list.
- ☐ **309.** Reverse a list in place, and also produce a reversed copy.
- ☐ **310.** Sort a list in ascending and descending order.
- ☐ **311.** Sort a list of strings ignoring case.
- ☐ **312.** Sort a list of strings by their length.
- ☐ **313.** Copy a list so that changes to the copy do not affect the original.
- ☐ **314.** Demonstrate the difference between shallow and deep copy with nested lists.
- ☐ **315.** Clear a list and check that it is empty.
- ☐ **316.** Create a list of numbers 1 to 20 using range.
- ☐ **317.** Slice a list to get the first three, last three and every other element.
- ☐ **318.** Replace a slice of a list with new elements.
- ☐ **319.** Delete a slice from a list.
- ☐ **320.** Join a list of numbers into a comma-separated string.

## List Operations

### STUDY THESE TOPICS FIRST

- Looping with index and enumerate
- min, max, sum without built-ins
- Two-pointer and swap techniques
- Sets for removing duplicates
- Merging sorted lists
- Rotating lists with slicing
- Hashing with dictionaries for pair problems
- Formula  $n(n+1)/2$  for missing numbers
- Recursion for flattening

- ☐ **321.** Find the sum, min and max of a list without using built-ins.
- ☐ **322.** Find the second largest number in a list.
- ☐ **323.** Find the k-th smallest number in a list.
- ☐ **324.** Find the average and median of a list.
- ☐ **325.** Find the mode of a list.
- ☐ **326.** Remove duplicates from a list while keeping the order.
- ☐ **327.** Find the common elements between two lists.
- ☐ **328.** Find elements present in the first list but not in the second.
- ☐ **329.** Merge two sorted lists into one sorted list.
- ☐ **330.** Reverse a list without using reverse() or slicing.
- ☐ **331.** Rotate a list left by k positions.
- ☐ **332.** Rotate a list right by k positions.
- ☐ **333.** Swap the first and last elements of a list.
- ☐ **334.** Move all zeros in a list to the end while keeping other order.
- ☐ **335.** Separate a list into even and odd numbers.
- ☐ **336.** Find the largest and smallest element positions in a list.
- ☐ **337.** Count positive, negative and zero values in a list.
- ☐ **338.** Find the cumulative sum of a list.
- ☐ **339.** Multiply every element of a list by 2 without a list comprehension.
- ☐ **340.** Find all pairs in a list whose sum equals a target.
- ☐ **341.** Find the missing number in a list of 1 to N.
- ☐ **342.** Find the duplicate number in a list.
- ☐ **343.** Check whether a list is sorted.
- ☐ **344.** Check whether two lists are identical regardless of order.
- ☐ **345.** Flatten a nested list of arbitrary depth.

## Nested Lists and Matrices

### STUDY THESE TOPICS FIRST

- Lists of lists and row/column indexing
- Creating matrices correctly (avoid `[[0]*n]*n` pitfall)
- Nested loops over rows and columns
- Matrix addition, multiplication and transpose rules
- `zip(*matrix)`
- Diagonals and index relationships
- Boundary-shrinking technique for spiral order

- ☐ **346.** Create a 3x3 matrix using nested lists and print it.
- ☐ **347.** Create a 5x5 identity matrix.
- ☐ **348.** Read a matrix from the user and print it nicely.
- ☐ **349.** Add two matrices.
- ☐ **350.** Subtract two matrices.
- ☐ **351.** Multiply two matrices.
- ☐ **352.** Transpose a matrix.
- ☐ **353.** Find the sum of each row and each column of a matrix.
- ☐ **354.** Find the sum of the diagonal elements of a square matrix.
- ☐ **355.** Check whether a matrix is symmetric.
- ☐ **356.** Rotate a square matrix 90 degrees clockwise.
- ☐ **357.** Print a matrix in spiral order.
- ☐ **358.** Find the largest element in each row of a matrix.
- ☐ **359.** Search for an element in a matrix and print its position.
- ☐ **360.** Print the upper triangular part of a matrix.
- ☐ **361.** Flatten a matrix into a single list.
- ☐ **362.** Compute the determinant of a 2x2 and 3x3 matrix.
- ☐ **363.** Implement a simple Tic-Tac-Toe board and check for a winner.

## Tuples

### STUDY THESE TOPICS FIRST

- Tuple creation, indexing, slicing and immutability
- Single-element tuple syntax
- Packing and unpacking, extended unpacking with `*`
- Tuples as dictionary keys
- `namedtuple` from `collections`
- `zip()` and `enumerate()`
- Returning multiple values from functions
- Sorting with `key=lambda`

- ☐ **364.** Create a tuple and access elements by index.
- ☐ **365.** Show that tuples are immutable by attempting an assignment.

- ☐ **366.** Convert a list to a tuple and a tuple to a list.
- ☐ **367.** Unpack a tuple into separate variables.
- ☐ **368.** Use extended unpacking with \* to split first, middle and last values.
- ☐ **369.** Swap two variables using tuple packing and unpacking.
- ☐ **370.** Concatenate and repeat tuples.
- ☐ **371.** Find the index and count of elements in a tuple.
- ☐ **372.** Create a tuple with a single element correctly.
- ☐ **373.** Sort a tuple and return the result as a tuple.
- ☐ **374.** Use a tuple as a dictionary key for coordinates.
- ☐ **375.** Create a list of tuples (name, age) and sort by age.
- ☐ **376.** Find the minimum and maximum in a list of tuples by the second element.
- ☐ **377.** Use namedtuple to represent a point with x and y.
- ☐ **378.** Return multiple values from a function as a tuple.
- ☐ **379.** Zip two lists into a list of tuples and unzip them back.
- ☐ **380.** Use enumerate on a list and start the index at 1.
- ☐ **381.** Show that a tuple containing a list is not fully immutable.

## List Comprehensions and Slicing Tricks

### STUDY THESE TOPICS FIRST

- List comprehension syntax with conditions
- Nested comprehensions
- Conditional expression inside a comprehension
- itertools: product, permutations, combinations
- Sliding window technique
- Kadane's algorithm idea
- The array module and an introduction to NumPy arrays

- ☐ **382.** Create a list of the squares of numbers 1 to 10 using a comprehension.
- ☐ **383.** Create a list of even numbers from 1 to 50 using a comprehension.
- ☐ **384.** Create a list of the lengths of words in a sentence.
- ☐ **385.** Convert a list of strings to uppercase with a comprehension.
- ☐ **386.** Filter a list of numbers to keep only primes using a comprehension.
- ☐ **387.** Create a list of tuples (n, n\*\*2) with a comprehension.
- ☐ **388.** Create a flat list from a matrix using a comprehension.
- ☐ **389.** Create a multiplication table as a nested comprehension.
- ☐ **390.** Use a conditional expression inside a comprehension (fizzbuzz style).
- ☐ **391.** Generate all pairs from two lists (Cartesian product) using a comprehension.
- ☐ **392.** Use itertools.product to generate the same pairs.
- ☐ **393.** Generate all permutations of a list using itertools.
- ☐ **394.** Generate all combinations of size 2 from a list.
- ☐ **395.** Split a list into chunks of size n.

- ☐ **396.** Find the maximum of each sliding window of size k in a list.
- ☐ **397.** Find the longest increasing subsequence length in a list.
- ☐ **398.** Find the maximum sum of a contiguous subarray (Kadane's algorithm).
- ☐ **399.** Use the array module to create an integer array and add elements.
- ☐ **400.** Use NumPy to create an array of 1 to 10 and compute its mean and standard deviation.

# Part 5: Dictionaries and Sets (Intermediate)

Questions 401 to 500

## Dictionary Basics

### STUDY THESE TOPICS FIRST

- Key-value pairs, creation and access
- `get()`, `setdefault()`, `update()`, `pop()`, `popitem()`, `del`
- `keys()`, `values()`, `items()`
- Iterating over dictionaries
- Dictionary merging with `|` and `update`
- `dict(zip())`, `dict.fromkeys()`
- Sorting with `sorted()` and `key`
- Dictionary comprehensions

- ☐ **401.** Create a dictionary of three students and their marks and print it.
- ☐ **402.** Access a value by key and by `get()` with a default.
- ☐ **403.** Add a new key-value pair and update an existing one.
- ☐ **404.** Delete a key using `del`, `pop` and `popitem`.
- ☐ **405.** Check whether a key exists in a dictionary.
- ☐ **406.** Print all keys, all values and all items of a dictionary.
- ☐ **407.** Loop through a dictionary and print key: value pairs.
- ☐ **408.** Merge two dictionaries using `update` and the `|` operator.
- ☐ **409.** Find the length of a dictionary.
- ☐ **410.** Copy a dictionary and modify the copy without changing the original.
- ☐ **411.** Create a dictionary from two lists using `zip`.
- ☐ **412.** Create a dictionary with keys from a list and a default value using `fromkeys`.
- ☐ **413.** Use `setdefault` to add a key only if it is missing.
- ☐ **414.** Sort a dictionary by key.
- ☐ **415.** Sort a dictionary by value in descending order.
- ☐ **416.** Find the key with the maximum value in a dictionary.
- ☐ **417.** Invert a dictionary (swap keys and values).
- ☐ **418.** Remove all keys whose value is `None` from a dictionary.
- ☐ **419.** Use a dictionary comprehension to create squares of numbers 1 to 10.
- ☐ **420.** Filter a dictionary to keep only items whose value is above a threshold.

## Dictionary Applications

### STUDY THESE TOPICS FIRST

- Counting patterns and Counter
- defaultdict, OrderedDict, ChainMap
- Nested dictionaries and safe access
- Grouping patterns
- Recursion for flattening nested data
- Memoization with dictionaries
- The json module basics

- ☐ **421.** Count the frequency of each word in a sentence using a dictionary.
- ☐ **422.** Count the frequency of each character using a dictionary.
- ☐ **423.** Count frequencies using collections.Counter and print the top 3.
- ☐ **424.** Group words by their first letter.
- ☐ **425.** Group words by their length.
- ☐ **426.** Build a phonebook that adds, searches and deletes contacts.
- ☐ **427.** Build a student database storing name, age and marks for several students.
- ☐ **428.** Find the student with the highest marks from a nested dictionary.
- ☐ **429.** Compute the average of each student's marks from a dictionary of lists.
- ☐ **430.** Create a nested dictionary and access an inner value safely.
- ☐ **431.** Flatten a nested dictionary into dot-separated keys.
- ☐ **432.** Unflatten a dot-separated dictionary into a nested one.
- ☐ **433.** Merge two dictionaries by adding values for shared keys.
- ☐ **434.** Find keys common to two dictionaries.
- ☐ **435.** Find the difference between two dictionaries.
- ☐ **436.** Convert a list of tuples to a dictionary.
- ☐ **437.** Convert a dictionary to a list of tuples sorted by value.
- ☐ **438.** Count how many times each number appears in a list and return those appearing more than once.
- ☐ **439.** Implement a simple inventory system with add, remove and low stock report.
- ☐ **440.** Use defaultdict(list) to group items by a category.
- ☐ **441.** Use defaultdict(int) to count items.
- ☐ **442.** Use OrderedDict to preserve and move keys to the end.
- ☐ **443.** Use ChainMap to look up values from multiple dictionaries.
- ☐ **444.** Implement a simple cache dictionary for function results.
- ☐ **445.** Store a dictionary as JSON in a string and read it back.

## Sets

### STUDY THESE TOPICS FIRST

- Set creation, uniqueness and unordered nature
- add, remove, discard, pop, clear
- Union, intersection, difference, symmetric difference
- Subset, superset and disjoint checks
- Set comprehensions
- frozenset
- Why membership tests are  $O(1)$

- ☐ **446.** Create a set from a list to remove duplicates.
- ☐ **447.** Add and remove elements from a set and compare discard with remove.
- ☐ **448.** Check membership in a set and explain why it is fast.
- ☐ **449.** Find the union of two sets in two different ways.
- ☐ **450.** Find the intersection of two sets.
- ☐ **451.** Find the difference and symmetric difference of two sets.
- ☐ **452.** Check whether one set is a subset or superset of another.
- ☐ **453.** Check whether two sets are disjoint.
- ☐ **454.** Find the length, min, max and sum of a set of numbers.
- ☐ **455.** Use a set comprehension to build a set of squares.
- ☐ **456.** Create a frozenset and use it as a dictionary key.
- ☐ **457.** Find the unique characters of a string using a set.
- ☐ **458.** Check whether a string has all unique characters.
- ☐ **459.** Find the elements that appear in exactly one of two lists.
- ☐ **460.** Check whether two lists have any common element.
- ☐ **461.** Find the first duplicate in a list using a set.
- ☐ **462.** Find the union of three or more sets using a loop.
- ☐ **463.** Find the number of unique words in a paragraph.
- ☐ **464.** Find the missing numbers in a list from a range using sets.
- ☐ **465.** Find the intersection of multiple lists.

## Mixed Data Structure Problems

### STUDY THESE TOPICS FIRST

- Hash map technique for lookups
- Sliding window with sets and dictionaries
- Sorting with key functions and itemgetter
- Counter and heapq.nlargest
- Lists of dictionaries as records
- OrderedDict and doubly-linked-list idea (LRU)
- Adjacency dictionary for graphs

- ☐ **466.** Two-sum: return indices of two numbers in a list that add up to a target using a dictionary.

- ☐ **467.** Check if two strings are isomorphic using a dictionary.
- ☐ **468.** Find the longest substring without repeating characters.
- ☐ **469.** Find the first unique character index in a string.
- ☐ **470.** Check whether a list contains duplicates within k indices.
- ☐ **471.** Find the majority element in a list.
- ☐ **472.** Find the top k most frequent elements in a list.
- ☐ **473.** Group a list of employee dictionaries by department.
- ☐ **474.** Sort a list of dictionaries by a given key.
- ☐ **475.** Sort a list of dictionaries by two keys.
- ☐ **476.** Find the employee with the highest salary from a list of dictionaries.
- ☐ **477.** Convert a list of dictionaries to a dictionary keyed by id.
- ☐ **478.** Count vowels in each word of a sentence and store as a dictionary.
- ☐ **479.** Build an index mapping each word to the line numbers where it appears.
- ☐ **480.** Create a simple Morse code translator using a dictionary.
- ☐ **481.** Implement a basic voting system that counts and shows the winner.
- ☐ **482.** Implement a shopping cart with prices, quantities and total.
- ☐ **483.** Build a student grade book that computes letter grades from marks.
- ☐ **484.** Implement a LRU cache using OrderedDict.
- ☐ **485.** Implement a simple graph as an adjacency dictionary and list the neighbours of a node.

## More Practice with Dictionaries and Sets

### STUDY THESE TOPICS FIRST

- Revise: dictionary methods and comprehensions
- Revise: set operations
- Canonical keys such as sorted(word)
- Recursion on nested dictionaries
- Hashability: why lists cannot be keys
- Using sets to find consecutive sequences

- ☐ **486.** Find all anagram pairs in a list of words using a dictionary of sorted keys.
- ☐ **487.** Find the intersection of two dictionaries' items.
- ☐ **488.** Count the number of distinct values in a dictionary.
- ☐ **489.** Find the keys that map to the same value in a dictionary.
- ☐ **490.** Create a dictionary of the first N Fibonacci numbers indexed by position.
- ☐ **491.** Create a dictionary mapping each letter to its position in the alphabet.
- ☐ **492.** Replace each word in a sentence using a replacement dictionary.
- ☐ **493.** Check whether two dictionaries are equal regardless of insertion order.
- ☐ **494.** Find the sum of all values in a nested dictionary of numbers.
- ☐ **495.** Find the depth of a nested dictionary.
- ☐ **496.** Remove duplicate dictionaries from a list of dictionaries.
- ☐ **497.** Find the symmetric difference of three sets.

- ☐ **498.** Check whether a list of numbers forms a consecutive sequence using a set.
- ☐ **499.** Find the longest consecutive sequence in an unsorted list using a set.
- ☐ **500.** Implement a simple spell checker that suggests words from a dictionary set.

# Part 6: Functions and Recursion (Intermediate)

Questions 501 to 600

---

## Function Basics

### STUDY THESE TOPICS FIRST

- def, parameters, arguments and return
- Return vs. print
- Returning multiple values
- Docstrings and help()
- Calling functions from other functions
- Pure functions and single responsibility
- Writing and testing small reusable functions

- ☐ **501.** Write a function that greets a person by name.
- ☐ **502.** Write a function that returns the sum of two numbers.
- ☐ **503.** Write a function that returns the larger of two numbers.
- ☐ **504.** Write a function that returns the factorial of a number.
- ☐ **505.** Write a function that checks whether a number is prime.
- ☐ **506.** Write a function that returns the list of primes up to N.
- ☐ **507.** Write a function that reverses a string.
- ☐ **508.** Write a function that checks whether a string is a palindrome.
- ☐ **509.** Write a function that counts vowels in a string.
- ☐ **510.** Write a function that returns both the minimum and maximum of a list.
- ☐ **511.** Write a function that converts Celsius to Fahrenheit and another that converts back.
- ☐ **512.** Write a function that returns the n-th Fibonacci number iteratively.
- ☐ **513.** Write a function that returns the sum of digits of a number.
- ☐ **514.** Write a function that returns the GCD of two numbers.
- ☐ **515.** Write a function that checks whether a year is a leap year.
- ☐ **516.** Write a function that returns True if all elements of a list are unique.
- ☐ **517.** Write a function with a docstring and print it using help() and \_\_doc\_\_.

## Arguments and Scope

### STUDY THESE TOPICS FIRST

- Default, keyword, positional arguments
- \*args and \*\*kwargs
- Positional-only (/) and keyword-only (\*) parameters
- Mutable default argument pitfall
- LEGB scope rule, global and nonlocal
- Closures
- Lambda, map, filter, reduce
- functools.partial
- Type hints
- any() and all()

- ☐ **518.** Write a function with a default argument value and call it with and without that argument.
- ☐ **519.** Call a function using keyword arguments in a different order.
- ☐ **520.** Write a function using \*args that returns the sum of any number of numbers.
- ☐ **521.** Write a function using \*\*kwargs that prints all key-value pairs it receives.
- ☐ **522.** Combine positional, \*args, keyword-only and \*\*kwargs in one function signature.
- ☐ **523.** Write a function that accepts positional-only parameters using the / syntax.
- ☐ **524.** Write a function that accepts keyword-only parameters using the \* syntax.
- ☐ **525.** Demonstrate the mutable default argument pitfall and fix it.
- ☐ **526.** Unpack a list into function arguments using \* and a dict using \*\*.
- ☐ **527.** Demonstrate local versus global variables with the global keyword.
- ☐ **528.** Demonstrate the nonlocal keyword in a nested function.
- ☐ **529.** Explain the LEGB rule with a short example.
- ☐ **530.** Show that mutable arguments are modified inside functions but immutable ones are not.
- ☐ **531.** Write a function with type hints and annotate its return value.
- ☐ **532.** Return multiple values from a function and unpack them.
- ☐ **533.** Write a nested function that returns another function (closure).
- ☐ **534.** Write a function that creates counters using closures.
- ☐ **535.** Write a lambda function that squares a number.
- ☐ **536.** Sort a list of tuples by the second item using a lambda.
- ☐ **537.** Use map with a lambda to cube each number in a list.
- ☐ **538.** Use filter with a lambda to keep numbers greater than 10.
- ☐ **539.** Use reduce from functools to compute the product of a list.
- ☐ **540.** Use functools.partial to create a function that always raises to the power 2.
- ☐ **541.** Use any() and all() with generator expressions to test a list.

## Recursion

### STUDY THESE TOPICS FIRST

- Base case and recursive case
- The call stack and RecursionError
- Recursion on numbers, strings and lists
- Memoization and `functools.lru_cache`
- Divide and conquer (binary search)
- Backtracking (subsets, permutations, N-Queens)
- Tower of Hanoi pattern

- ☐ **542.** Write a recursive function to compute the factorial of N.
- ☐ **543.** Write a recursive function for the N-th Fibonacci number.
- ☐ **544.** Make the recursive Fibonacci faster using memoization with a dictionary.
- ☐ **545.** Make the recursive Fibonacci faster using `functools.lru_cache`.
- ☐ **546.** Write a recursive function to compute the sum of the first N natural numbers.
- ☐ **547.** Write a recursive function to compute the sum of digits.
- ☐ **548.** Write a recursive function to reverse a string.
- ☐ **549.** Write a recursive function to check whether a string is a palindrome.
- ☐ **550.** Write a recursive function to compute  $a^{**}b$ .
- ☐ **551.** Write a recursive function to find the GCD of two numbers.
- ☐ **552.** Write a recursive function to print numbers from N down to 1.
- ☐ **553.** Write a recursive function to find the maximum value in a list.
- ☐ **554.** Write a recursive function to compute the sum of a list.
- ☐ **555.** Write a recursive function to flatten a nested list.
- ☐ **556.** Write a recursive function to convert a decimal number to binary.
- ☐ **557.** Write a recursive binary search function.
- ☐ **558.** Solve the Tower of Hanoi for N disks and print the moves.
- ☐ **559.** Generate all subsets of a list recursively.
- ☐ **560.** Generate all permutations of a string recursively.
- ☐ **561.** Generate all balanced parentheses combinations for N pairs.
- ☐ **562.** Solve the N-Queens problem for  $N = 4$  and print one solution.
- ☐ **563.** Find all paths from top-left to bottom-right in a grid using recursion.
- ☐ **564.** Find the recursion limit using `sys` and change it safely, then show a `RecursionError`.

## Functional Practice

### STUDY THESE TOPICS FIRST

- Higher-order functions
- Function composition and currying
- Generators (introduction)
- `time.perf_counter()`
- Decorator idea (functions that wrap functions)
- Binary search on answers
- Number to words logic

- ☐ **565.** Write a function that applies another function to each element of a list.
- ☐ **566.** Write a function that composes two functions into one.
- ☐ **567.** Write a function that returns a list of the first N squares using a generator.
- ☐ **568.** Implement your own versions of `map` and `filter`.
- ☐ **569.** Implement a function that curries a two-argument function.
- ☐ **570.** Write a function that counts how many times it has been called using a function attribute.
- ☐ **571.** Write a function that times how long another function takes using `time.perf_counter`.
- ☐ **572.** Write a function that retries another function up to 3 times when it raises an error.
- ☐ **573.** Write a function that takes a list of numbers and returns a dictionary of statistics.
- ☐ **574.** Write a function that validates a password and returns a list of failed rules.
- ☐ **575.** Write a function that converts seconds to a readable string such as "1h 2m 3s".
- ☐ **576.** Write a function that returns all divisors of a number.
- ☐ **577.** Write a function that checks whether a number is a perfect square without using `sqrt`.
- ☐ **578.** Write a function that returns the next prime number after N.
- ☐ **579.** Write a function that finds the integer square root using binary search.
- ☐ **580.** Write a function that converts a number to words (up to 999).
- ☐ **581.** Write a function that multiplies two matrices passed as lists.
- ☐ **582.** Write a function that computes the median of a list without modifying the list.

## Mini Programs Using Functions

### STUDY THESE TOPICS FIRST

- Breaking a program into functions
- Menu-driven programs with `while True`
- Dictionaries mapping choices to functions
- `random` module for games
- Input validation with `try/except`
- Game state in lists and dictionaries
- The `calendar` module

- ☐ **583.** Build a calculator program with separate functions for each operation.
- ☐ **584.** Build a temperature converter with a menu.
- ☐ **585.** Build a number guessing game using functions.

- ☐ **586.** Build a rock-paper-scissors game against the computer.
- ☐ **587.** Build a simple quiz game with a list of questions stored in a dictionary.
- ☐ **588.** Build a to-do list manager with add, view and delete functions.
- ☐ **589.** Build a bank account simulation using functions and a dictionary.
- ☐ **590.** Build a tip and bill splitting calculator.
- ☐ **591.** Build a prime number generator with a menu of options.
- ☐ **592.** Build a simple password generator with options for length and character types.
- ☐ **593.** Build a Hangman game using functions.
- ☐ **594.** Build a unit converter for length, weight and temperature.
- ☐ **595.** Build an EMI calculator for a loan.
- ☐ **596.** Build a simple text-based menu using a dictionary that maps choices to functions.
- ☐ **597.** Build a simple Tic-Tac-Toe game for two players.
- ☐ **598.** Build a number-to-Roman-numeral converter as a function.
- ☐ **599.** Build a function that prints a calendar for a given month using calendar module.
- ☐ **600.** Build a word counter function that reads text and returns counts.

# Part 7: Files, Exceptions and Modules (Intermediate)

Questions 601 to 700

## Reading and Writing Files

### STUDY THESE TOPICS FIRST

- open() modes: r, w, a, r+, rb, wb
- The with statement (context manager)
- read(), readline(), readlines(), iterating a file
- write() and writelines()
- File pointer: seek() and tell()
- os.path and pathlib
- Binary files
- Encoding (utf-8)

- ☐ **601.** Write three lines of text to a file named notes.txt.
- ☐ **602.** Read the entire contents of a file and print it.
- ☐ **603.** Read a file line by line and print each line without the trailing newline.
- ☐ **604.** Read a file and print the number of lines, words and characters.
- ☐ **605.** Append a new line to an existing file.
- ☐ **606.** Read the first N lines of a file.
- ☐ **607.** Read the last N lines of a file.
- ☐ **608.** Print the longest line in a file.
- ☐ **609.** Count the occurrences of a given word in a file.
- ☐ **610.** Find and print all lines containing a given keyword.
- ☐ **611.** Copy the contents of one file to another.
- ☐ **612.** Merge the contents of two files into a third file.
- ☐ **613.** Reverse the order of lines in a file and save it.
- ☐ **614.** Remove blank lines from a file.
- ☐ **615.** Convert the contents of a file to uppercase and save it.
- ☐ **616.** Replace a word in a file with another word throughout.
- ☐ **617.** Write a list of numbers to a file, one per line, and read them back as integers.
- ☐ **618.** Read a file and write only the even-numbered lines to a new file.
- ☐ **619.** Check whether a file exists before reading it using os.path and pathlib.
- ☐ **620.** Use the with statement and explain why it is better than manual close().
- ☐ **621.** Read a large file in chunks of fixed size.
- ☐ **622.** Use seek() and tell() to move around in a file.
- ☐ **623.** Read and write a binary file such as an image copy.
- ☐ **624.** Use pathlib to read and write a text file.
- ☐ **625.** Count the frequency of each word in a file and write the result to a new file.

## CSV, JSON and Other Formats

### STUDY THESE TOPICS FIRST

- The csv module: reader, writer, DictReader, DictWriter
- The json module: load, loads, dump, dumps
- pickle and shelve
- configparser and xml.etree
- SQLite with sqlite3: connect, cursor, execute, commit
- Parameterised queries
- Introduction to pandas read\_csv and to\_csv

- ☐ **626.** Write a list of dictionaries to a CSV file using csv.DictWriter.
- ☐ **627.** Read a CSV file and print each row using csv.reader.
- ☐ **628.** Read a CSV file with DictReader and compute the average of a numeric column.
- ☐ **629.** Filter rows of a CSV by a condition and save to a new CSV.
- ☐ **630.** Sort the rows of a CSV file by a column and write them back.
- ☐ **631.** Convert a CSV file to JSON.
- ☐ **632.** Convert a JSON file to CSV.
- ☐ **633.** Write a Python dictionary to a JSON file with indentation.
- ☐ **634.** Read a JSON file and print a nested value.
- ☐ **635.** Serialize and deserialize a Python object using pickle.
- ☐ **636.** Use json.dumps with sort\_keys and a custom default function for dates.
- ☐ **637.** Read a configuration file using configparser.
- ☐ **638.** Parse a simple XML string using xml.etree.ElementTree.
- ☐ **639.** Read a text file with a different encoding and handle decode errors.
- ☐ **640.** Use the shelve module to store key-value data persistently.
- ☐ **641.** Create a SQLite database, create a table and insert three rows.
- ☐ **642.** Query a SQLite table and print rows with a WHERE condition.
- ☐ **643.** Update and delete rows in a SQLite table using parameterised queries.
- ☐ **644.** Read a SQLite table into a pandas DataFrame.
- ☐ **645.** Write a DataFrame to a CSV file and read it back.

## Exceptions

### STUDY THESE TOPICS FIRST

- try, except, else, finally flow
- Common exceptions: ValueError, TypeError, KeyError, IndexError, ZeroDivisionError, FileNotFoundError
- Catching multiple exceptions
- raise and re-raise, raise from
- Custom exception classes
- assert
- Context managers and contextlib
- warnings module
- ExceptionGroup and except\*

- ☐ **646.** Handle `ZeroDivisionError` when dividing two numbers.
- ☐ **647.** Handle `ValueError` when converting user input to an integer.
- ☐ **648.** Handle `FileNotFoundError` when opening a file that does not exist.
- ☐ **649.** Handle `IndexError` when accessing a list out of range.
- ☐ **650.** Handle `KeyError` when accessing a missing dictionary key.
- ☐ **651.** Handle multiple exceptions in one `except` clause using a tuple.
- ☐ **652.** Use separate `except` clauses for different exceptions.
- ☐ **653.** Use `try`, `except`, `else` and `finally` together and explain the order of execution.
- ☐ **654.** Catch any exception using `Exception` and print its message.
- ☐ **655.** Print the traceback of an exception using the `traceback` module.
- ☐ **656.** Raise a `ValueError` manually if an age is negative.
- ☐ **657.** Re-raise an exception after logging it.
- ☐ **658.** Chain exceptions using `raise ... from`.
- ☐ **659.** Create a custom exception class and raise it.
- ☐ **660.** Create a custom exception hierarchy for a banking application.
- ☐ **661.** Use `assert` statements to validate function inputs.
- ☐ **662.** Write a function that keeps asking for input until a valid integer is entered.
- ☐ **663.** Write a safe division function that returns `None` on error.
- ☐ **664.** Use `contextlib.suppress` to ignore a specific exception.
- ☐ **665.** Write a context manager class that prints on entering and exiting.
- ☐ **666.** Write a context manager with the `contextlib.contextmanager` decorator.
- ☐ **667.** Use the `warnings` module to emit a deprecation warning.
- ☐ **668.** Handle `KeyboardInterrupt` in a long running loop.
- ☐ **669.** Use `ExceptionGroup` and `except*` to handle multiple errors.
- ☐ **670.** Write a retry decorator-free loop that retries a file read three times with delays.

## Modules and Packages

### STUDY THESE TOPICS FIRST

- `import`, `from ... import`, `as`
- Creating your own modules
- `__name__ == '__main__'`
- Packages and `__init__.py`, relative imports
- `sys.path` and `sys.argv`
- `argparse`
- `os`, `shutil`, `glob`, `tempfile`, `subprocess`
- Virtual environments, `pip` and `requirements.txt`
- `logging` module

- ☐ **671.** Import a module and use one of its functions.
- ☐ **672.** Import specific functions with `from ... import` and rename them using `as`.
- ☐ **673.** Create your own module with two functions and import it.
- ☐ **674.** Use the `__name__ == "__main__"` guard and explain why it is useful.

- ☐ **675.** List the contents of a module using `dir()`.
- ☐ **676.** Find where a module is located using its `__file__` attribute.
- ☐ **677.** Create a package with `__init__.py` and two modules and import from it.
- ☐ **678.** Use relative imports inside a package.
- ☐ **679.** Print `sys.path` and add a custom directory to it.
- ☐ **680.** Read command-line arguments using `sys.argv`.
- ☐ **681.** Build a command-line tool using `argparse` with options and flags.
- ☐ **682.** Use `os` to list files in a directory and print their sizes.
- ☐ **683.** Use `os.walk` to find all `.py` files in a directory tree.
- ☐ **684.** Use `shutil` to copy, move and delete files.
- ☐ **685.** Use `glob` to find all text files matching a pattern.
- ☐ **686.** Create and delete a temporary file using `tempfile`.
- ☐ **687.** Use the `platform` and `os` modules to print system information.
- ☐ **688.** Read environment variables using `os.environ` and `os.getenv`.
- ☐ **689.** Use `subprocess` to run a shell command and capture its output.
- ☐ **690.** Install a third-party package using `pip` and use it in a script.
- ☐ **691.** Create and activate a virtual environment and list installed packages.
- ☐ **692.** Write a `requirements.txt` for a small project.
- ☐ **693.** Use the logging module to write messages at different levels to a file.
- ☐ **694.** Configure logging with a format that includes time, level and message.
- ☐ **695.** Use the `collections`, `itertools` and `functools` modules in one small program.

## Extra File and Module Practice

### STUDY THESE TOPICS FIRST

- Revise: `os`, `pathlib` and `shutil`
- `hashlib` (`md5`, `sha256`)
- Reading files in chunks
- Dictionary grouping by extension

- ☐ **696.** Write a program that organizes files in a folder into subfolders by extension.
- ☐ **697.** Write a program that renames all files in a folder with a numbered prefix.
- ☐ **698.** Find the largest file in a directory tree.
- ☐ **699.** Find duplicate files in a folder by comparing their hashes with `hashlib`.
- ☐ **700.** Compute the MD5 and SHA-256 hash of a string and of a file.

# Part 8: Object-Oriented Programming (Intermediate to Advanced)

Questions 701 to 800

## Classes and Objects

### STUDY THESE TOPICS FIRST

- class, object, `__init__` and `self`
- Instance vs. class variables
- Methods and attributes
- `__str__` and `__repr__`
- Operator overloading dunder methods (`__add__`, `__eq__`, `__lt__`, `__len__`, `__getitem__`, `__call__`, `__bool__`, `__iter__`)
- Context manager methods `__enter__` and `__exit__`
- `__slots__`
- dataclasses

- ☐ **701.** Create a class `Person` with name and age attributes and print them.
- ☐ **702.** Add a method to the `Person` class that introduces the person.
- ☐ **703.** Create a class `Rectangle` with methods for area and perimeter.
- ☐ **704.** Create a class `Circle` with a class attribute for `pi`.
- ☐ **705.** Create a class `Student` that stores marks and computes the average and grade.
- ☐ **706.** Create a class `Counter` that increments, decrements and resets.
- ☐ **707.** Create a class `BankAccount` with deposit, withdraw and balance methods.
- ☐ **708.** Prevent a `BankAccount` from going negative by raising an exception.
- ☐ **709.** Create a class `Car` with attributes and a method that updates the mileage.
- ☐ **710.** Create a class `Book` and a class `Library` that stores many books.
- ☐ **711.** Demonstrate the difference between instance variables and class variables.
- ☐ **712.** Count the number of objects created from a class using a class variable.
- ☐ **713.** Use `__str__` and `__repr__` to control how an object prints.
- ☐ **714.** Use `__eq__` and `__hash__` so that two objects with the same data are equal.
- ☐ **715.** Use `__lt__` and `functools.total_ordering` to make objects sortable.
- ☐ **716.** Use `__len__` and `__getitem__` to make a class behave like a sequence.
- ☐ **717.** Use `__add__` to add two `Vector` objects.
- ☐ **718.** Use `__mul__` and `__rmul__` to scale a `Vector` by a number.
- ☐ **719.** Use `__call__` so that an object can be called like a function.
- ☐ **720.** Use `__bool__` to define the truthiness of an object.
- ☐ **721.** Use `__iter__` and `__next__` to make a class iterable.
- ☐ **722.** Use `__enter__` and `__exit__` to make a class a context manager.
- ☐ **723.** Use `__contains__` to support the `in` operator for a custom collection.
- ☐ **724.** Use `__slots__` to reduce memory usage and show the effect.
- ☐ **725.** Use dataclasses to create a `Point` class with default values.

## Encapsulation, Properties and Methods

### STUDY THESE TOPICS FIRST

- Public, protected (`_x`) and private (`__x`) names, name mangling
- `@property`, setters and deleters
- `@classmethod` and `@staticmethod`
- `getattr`, `setattr`, `hasattr`, `delattr`, `vars()`
- frozen dataclasses and `NamedTuple`
- `functools.cached_property`
- Singleton and `__new__`
- Data structure basics: stack, queue, linked list, BST

- ☐ **726.** Use a single underscore and a double underscore attribute and show name mangling.
- ☐ **727.** Use the `@property` decorator to create a read-only attribute.
- ☐ **728.** Use a property setter to validate that age cannot be negative.
- ☐ **729.** Use a property deleter to clean up an attribute.
- ☐ **730.** Use a `@classmethod` as an alternative constructor.
- ☐ **731.** Use a `@staticmethod` for a utility function inside a class.
- ☐ **732.** Explain with code when to choose instance, class and static methods.
- ☐ **733.** Create a class with a private method and call it from a public method.
- ☐ **734.** Use `getattr`, `setattr`, `hasattr` and `delattr` on an object.
- ☐ **735.** Print all attributes of an object using `vars()` and `__dict__`.
- ☐ **736.** Add an attribute to an object dynamically and show it is not on the class.
- ☐ **737.** Make a class immutable using frozen dataclasses.
- ☐ **738.** Use `field(default_factory=list)` in a dataclass.
- ☐ **739.** Use `__post_init__` in a dataclass for validation.
- ☐ **740.** Use `NamedTuple` from typing to define a record.
- ☐ **741.** Implement a Temperature class that stores Celsius and exposes Fahrenheit as a property.
- ☐ **742.** Implement a lazily computed property using `functools.cached_property`.
- ☐ **743.** Implement a Singleton class using `__new__`.
- ☐ **744.** Implement a class that prevents setting unknown attributes.
- ☐ **745.** Implement a Stack class with push, pop, peek and `is_empty`.
- ☐ **746.** Implement a Queue class using a list and then using `collections.deque`.
- ☐ **747.** Implement a Node and LinkedList class with insert, delete and search.
- ☐ **748.** Implement a doubly linked list with forward and backward traversal.
- ☐ **749.** Implement a binary search tree class with insert and in-order traversal.
- ☐ **750.** Implement a Matrix class that supports addition and multiplication through operators.

## Inheritance and Polymorphism

### STUDY THESE TOPICS FIRST

- Single, multiple and multilevel inheritance
- `super()` and method overriding
- Method Resolution Order (MRO)
- Mixins
- `isinstance()` and `issubclass()`
- Abstract base classes (abc)
- Duck typing and `typing.Protocol`
- Composition vs. inheritance
- `__init_subclass__`

- ☐ 751. Create a class `Animal` and a subclass `Dog` that overrides `speak()`.
- ☐ 752. Use `super()` to call a parent constructor from a subclass.
- ☐ 753. Create a class hierarchy `Employee`, `Manager` and `Developer` with salary logic.
- ☐ 754. Demonstrate method overriding and calling the parent version using `super()`.
- ☐ 755. Demonstrate multiple inheritance and print the Method Resolution Order.
- ☐ 756. Solve the diamond problem and explain the MRO result.
- ☐ 757. Use a mixin class to add logging behaviour to another class.
- ☐ 758. Use `isinstance` and `issubclass` to check types in a hierarchy.
- ☐ 759. Create an abstract base class `Shape` with abstract `area()` using `abc`.
- ☐ 760. Create `Circle` and `Square` subclasses of `Shape` and compute areas in a list loop.
- ☐ 761. Show polymorphism by calling the same method on objects of different classes.
- ☐ 762. Show duck typing with two unrelated classes having the same method.
- ☐ 763. Use composition: create a `Car` class that contains an `Engine` object.
- ☐ 764. Explain with code the difference between "is-a" and "has-a" relationships.
- ☐ 765. Override `__str__` in a subclass and call `super().__str__()`.
- ☐ 766. Use class methods to build subclasses through a factory.
- ☐ 767. Implement an abstract class with an abstract property.
- ☐ 768. Use `typing.Protocol` to define a structural interface.
- ☐ 769. Prevent a class from being subclassed using `__init_subclass__` or `final`.
- ☐ 770. Register subclasses automatically using `__init_subclass__`.

## Advanced OOP and Design Patterns

### STUDY THESE TOPICS FIRST

- Design patterns: Factory, Observer, Strategy, Decorator, Command, Iterator, Builder, Adapter, State, Registry
- Metaclasses
- Descriptors
- Enum, auto, Flag
- weakref and garbage collection
- `__getattr__`
- UML-style design of larger systems

- ☐ **771.** Implement the Factory pattern to create shapes from a string name.
- ☐ **772.** Implement the Observer pattern with subscribers and notifications.
- ☐ **773.** Implement the Strategy pattern for different discount calculations.
- ☐ **774.** Implement the Decorator pattern for adding toppings to a coffee order.
- ☐ **775.** Implement the Command pattern for undo and redo of text edits.
- ☐ **776.** Implement the Iterator pattern for a custom tree traversal.
- ☐ **777.** Implement the Builder pattern for constructing a complex object step by step.
- ☐ **778.** Implement the Adapter pattern for a legacy interface.
- ☐ **779.** Implement the State pattern for a traffic light.
- ☐ **780.** Implement the Registry pattern using a class decorator.
- ☐ **781.** Create a metaclass that adds an attribute to every class it creates.
- ☐ **782.** Create a descriptor that validates integer values within a range.
- ☐ **783.** Create a class that counts its method calls using a decorator.
- ☐ **784.** Create a simple Enum for days of the week and iterate over it.
- ☐ **785.** Use Enum with `auto()` and Flag for permission bits.
- ☐ **786.** Use weakref to hold a reference without preventing garbage collection.
- ☐ **787.** Show how reference counting works using `sys.getrefcount` and the `gc` module.
- ☐ **788.** Implement `__getattr__` to provide default values for missing attributes.
- ☐ **789.** Implement an object-oriented parking lot system.
- ☐ **790.** Implement an object-oriented library management system with members and books.
- ☐ **791.** Implement an object-oriented ATM simulation with accounts and transactions.
- ☐ **792.** Implement an object-oriented tic-tac-toe game with Player and Board classes.
- ☐ **793.** Implement a simple inventory system using inheritance and polymorphism.
- ☐ **794.** Implement an event system where classes register callbacks for named events.
- ☐ **795.** Implement a text adventure game engine with Room and Item classes.

## Extra OOP Practice

### STUDY THESE TOPICS FIRST

- Revise: operator overloading
- Revise: inheritance and polymorphism
- Strategy pattern for payroll and discounts
- Iterable and `__iter__` for collections

- ☐ **796.** Implement a Fraction class with addition, subtraction and simplification.
- ☐ **797.** Implement a Money class that supports comparison and arithmetic between the same currencies.
- ☐ **798.** Implement a Playlist class that supports iteration, adding and shuffling songs.
- ☐ **799.** Implement a ShoppingCart class with discount and tax strategies.
- ☐ **800.** Implement a Employee payroll system where each type calculates pay differently.

# Part 9: Intermediate to Advanced Python Features

Questions 801 to 900

## Iterators and Generators

### STUDY THESE TOPICS FIRST

- Iterable vs. iterator, iter() and next()
- StopIteration
- Generator functions and yield
- Generator expressions
- yield from and send()
- Lazy evaluation and memory benefits
- itertools: islice, chain, cycle, groupby, accumulate, tee, starmap

- ☐ **801.** Write a generator function that yields numbers from 1 to N.
- ☐ **802.** Write a generator that yields the Fibonacci sequence indefinitely.
- ☐ **803.** Write a generator that yields prime numbers one at a time.
- ☐ **804.** Write a generator that reads a large file line by line.
- ☐ **805.** Write a generator expression to sum the squares of the first million numbers and compare memory with a list.
- ☐ **806.** Use next() on an iterator and handle StopIteration.
- ☐ **807.** Create a custom iterator class that counts down from N to 1.
- ☐ **808.** Create an iterator that cycles through a list a given number of times.
- ☐ **809.** Use yield from to delegate to a sub-generator.
- ☐ **810.** Use send() to pass values into a generator.
- ☐ **811.** Write a generator pipeline that reads, filters and transforms data lazily.
- ☐ **812.** Use itertools.islice to take the first N items from an infinite generator.
- ☐ **813.** Use itertools.chain, cycle, repeat and count in short examples.
- ☐ **814.** Use itertools.groupby to group sorted records by a key.
- ☐ **815.** Use itertools.accumulate to compute running totals and running maximums.
- ☐ **816.** Use itertools.combinations\_with\_replacement for dice rolls.
- ☐ **817.** Use itertools.zip\_longest and itertools.starmap together.
- ☐ **818.** Use itertools.tee to iterate over the same iterator twice.
- ☐ **819.** Write a generator that produces all permutations of a list without using itertools.
- ☐ **820.** Write a generator that batches an iterable into groups of size n.

## Decorators and Closures

### STUDY THESE TOPICS FIRST

- First-class functions and closures
- Writing a basic decorator
- \*args and \*\*kwargs in wrappers
- functools.wraps
- Decorators with arguments
- Class-based decorators
- Stacking decorators
- functools.singledispatch and lru\_cache

- ☐ **821.** Write a decorator that prints "Start" and "End" around a function call.
- ☐ **822.** Write a decorator that measures and prints the execution time of a function.
- ☐ **823.** Write a decorator that logs the function name, arguments and return value.
- ☐ **824.** Write a decorator that works on functions with any arguments using \*args and \*\*kwargs.
- ☐ **825.** Use functools.wraps to preserve the name and docstring of the wrapped function.
- ☐ **826.** Write a decorator that takes arguments, such as repeat(n).
- ☐ **827.** Write a decorator that caches results of a function (your own memoize).
- ☐ **828.** Write a decorator that retries a function on failure with a delay.
- ☐ **829.** Write a decorator that checks that all arguments are of type int.
- ☐ **830.** Write a decorator that restricts a function to being called only once.
- ☐ **831.** Write a decorator that limits how often a function can be called per second.
- ☐ **832.** Stack two decorators on one function and explain the order of execution.
- ☐ **833.** Write a class-based decorator using \_\_call\_\_.
- ☐ **834.** Write a decorator that registers functions in a global dictionary.
- ☐ **835.** Write a decorator for methods that logs the instance's class name.
- ☐ **836.** Write a decorator that converts the return value of a function to JSON.
- ☐ **837.** Write a decorator that makes a function run in a background thread.
- ☐ **838.** Create a timeout decorator using signal or threads.
- ☐ **839.** Use functools.singledispatch to overload a function by argument type.
- ☐ **840.** Use functools.lru\_cache with maxsize and inspect its cache\_info.

## Regular Expressions

### STUDY THESE TOPICS FIRST

- The re module: search, match, fullmatch, findall, finditer, sub, split, compile
- Character classes, anchors and quantifiers
- Groups and named groups
- Greedy vs. non-greedy matching
- Lookahead and lookbehind
- Flags: IGNORECASE, MULTILINE, VERBOSE

- ☐ **841.** Check whether a string contains a number using re.search.
- ☐ **842.** Extract all numbers from a string using re.findall.
- ☐ **843.** Validate an email address using a regular expression.
- ☐ **844.** Validate an Indian mobile number using a regular expression.
- ☐ **845.** Validate a date in the format DD/MM/YYYY.
- ☐ **846.** Extract all hashtags and mentions from a tweet-like string.
- ☐ **847.** Extract all URLs from a block of text.
- ☐ **848.** Replace multiple spaces with a single space using re.sub.
- ☐ **849.** Use re.sub with a function to transform each match.
- ☐ **850.** Split a string on multiple delimiters using re.split.
- ☐ **851.** Use named groups to parse a log line into fields.
- ☐ **852.** Use re.compile and flags like IGNORECASE and MULTILINE.
- ☐ **853.** Use greedy versus non-greedy quantifiers on HTML-like text.
- ☐ **854.** Use lookahead and lookbehind assertions to find numbers after a currency symbol.
- ☐ **855.** Check password strength using multiple regular expressions.
- ☐ **856.** Find all words that start with a capital letter.
- ☐ **857.** Find repeated words such as "the the" in a sentence.
- ☐ **858.** Mask all digits in a string except the last 4 using regex.
- ☐ **859.** Parse an Apache-style log file and count requests per IP address.
- ☐ **860.** Write a tokenizer for simple arithmetic expressions with regular expressions.

## Standard Library and Productivity Tools

### STUDY THESE TOPICS FIRST

- collections: deque, Counter, defaultdict
- heapq and bisect
- datetime, zoneinfo and calendar
- secrets, statistics, fractions
- typing module
- operator module
- zipfile, gzip, tarfile
- urllib and the requests library
- contextlib.ExitStack

- ☐ **861.** Use `collections.deque` to implement a sliding window.
- ☐ **862.** Use `heapq` to find the 5 largest and 5 smallest values in a list.
- ☐ **863.** Implement a priority queue using `heapq` with tuple priorities.
- ☐ **864.** Use `bisect` to insert items into a sorted list and to find positions.
- ☐ **865.** Use `datetime` and `timedelta` to compute a date 100 days from today.
- ☐ **866.** Parse a date string with `strptime` and format it with `strftime`.
- ☐ **867.** Convert between time zones using `zoneinfo`.
- ☐ **868.** Use the `calendar` module to print the calendar of a year.
- ☐ **869.** Generate a UUID and a secure random token using `secrets`.
- ☐ **870.** Use `statistics` to compute mean, median, mode, variance and standard deviation.
- ☐ **871.** Use `fractions.Fraction` to do exact arithmetic with rational numbers.
- ☐ **872.** Use `textwrap`, `pprint` and `string.Template` in one short program.
- ☐ **873.** Use typing features such as `Optional`, `Union`, `TypedDict` and `Literal`.
- ☐ **874.** Use `enum.IntEnum` to compare enum members with integers.
- ☐ **875.** Use `operator.itemgetter` and `attrgetter` to sort complex data.
- ☐ **876.** Use `contextlib.ExitStack` to open several files dynamically.
- ☐ **877.** Use `zipfile` to create and extract a ZIP archive.
- ☐ **878.** Use `gzip` and `tarfile` to compress and read files.
- ☐ **879.** Use `urllib` to fetch the text of a web page and count its words.
- ☐ **880.** Use the `requests` library to call a public API and parse the JSON result.

## Pythonic Idioms and Debugging

### STUDY THESE TOPICS FIRST

- PEP 8 style and linters (`flake8`, `ruff`)
- EAFP vs. LBYL
- Unpacking and `enumerate/zip` idioms
- Debugging with `print`, logging, `pdb` and `breakpoint()`
- `timeit`, `cProfile`, `tracemalloc`
- `dis` and `sys.getsizeof`
- Structural pattern matching
- `doctest` and type checking with `mypy`
- Refactoring

- ☐ **881.** Rewrite a C-style index loop into a Pythonic loop using `enumerate` and `zip`.
- ☐ **882.** Use unpacking to swap, split and rebuild lists in one statement.
- ☐ **883.** Use the `any/all` idiom instead of flag variables.
- ☐ **884.** Use EAFP versus LBYL in a dictionary lookup example.
- ☐ **885.** Use the sentinel pattern for a function with an optional list argument.
- ☐ **886.** Use f-string debugging with the `=` specifier.
- ☐ **887.** Use `pdb` or `breakpoint()` to step through a buggy function.
- ☐ **888.** Use `assert` and logging to trace the flow of a recursive function.
- ☐ **889.** Use `timeit` to compare two ways of building a string.

- ☐ **890.** Use cProfile to find the slowest function in a program.
- ☐ **891.** Use tracemalloc to find the line allocating the most memory.
- ☐ **892.** Use sys.getsizeof to compare the sizes of a list, tuple, set and dict.
- ☐ **893.** Use dis to inspect the bytecode of a simple function.
- ☐ **894.** Use the walrus operator to simplify a while loop reading chunks.
- ☐ **895.** Use structural pattern matching to parse commands like "go north" or "take lamp".
- ☐ **896.** Use match with class patterns and guard conditions.
- ☐ **897.** Write doctests for a function and run them with the doctest module.
- ☐ **898.** Write a small program following PEP 8 and check it with a linter like flake8 or ruff.
- ☐ **899.** Add type hints to a module and check it with mypy.
- ☐ **900.** Refactor a 40-line function into smaller well-named functions.

# Part 10: Advanced Algorithms, Concurrency and Projects (Advanced)

Questions 901 to 1000

---

## Searching, Sorting and Complexity

### STUDY THESE TOPICS FIRST

- Big-O notation (time and space)
- Linear and binary search
- Bubble, selection, insertion, merge, quick, heap, counting and radix sort
- Recursion and divide and conquer
- bisect and heapq
- Sieve of Eratosthenes
- Empirical timing with timeit

- ☐ **901.** Implement linear search and binary search and compare their time complexity.
- ☐ **902.** Implement bubble sort and count the number of swaps.
- ☐ **903.** Implement selection sort and insertion sort.
- ☐ **904.** Implement merge sort recursively.
- ☐ **905.** Implement quick sort with a random pivot.
- ☐ **906.** Implement heap sort using a manual heap.
- ☐ **907.** Implement counting sort for small integer ranges.
- ☐ **908.** Implement radix sort for non-negative integers.
- ☐ **909.** Compare the running time of three sorting algorithms on 10,000 random numbers.
- ☐ **910.** Find the k-th largest element using quickselect.
- ☐ **911.** Search a rotated sorted array in  $O(\log n)$ .
- ☐ **912.** Find the first and last position of a target in a sorted list using bisect.
- ☐ **913.** Find the peak element in a list in  $O(\log n)$ .
- ☐ **914.** Find the square root of a number to 6 decimals using binary search.
- ☐ **915.** Count inversions in a list using merge sort.
- ☐ **916.** Merge k sorted lists using a heap.
- ☐ **917.** Find the median of two sorted lists.
- ☐ **918.** Implement the Sieve of Eratosthenes to find all primes below one million.
- ☐ **919.** Implement fast exponentiation and use it to compute Fibonacci numbers by matrix power.
- ☐ **920.** Estimate the time complexity of a function empirically by plotting runtimes for growing n.

## Data Structures from Scratch

### STUDY THESE TOPICS FIRST

- Stacks, queues and deques
- Linked lists
- Hash tables and collision handling
- Heaps and priority queues
- Trees: binary, BST, AVL, tries
- Traversals (pre, in, post, level order)
- Union-find, segment trees, Fenwick trees
- Skip lists and Bloom filters

- ☐ **921.** Implement a stack using two queues.
- ☐ **922.** Implement a queue using two stacks.
- ☐ **923.** Implement a min-stack that returns the minimum in  $O(1)$ .
- ☐ **924.** Implement a circular queue with a fixed size.
- ☐ **925.** Implement a hash map with separate chaining.
- ☐ **926.** Implement a hash map with open addressing and resizing.
- ☐ **927.** Implement a min-heap and a max-heap class.
- ☐ **928.** Implement a trie with insert, search and prefix search.
- ☐ **929.** Implement an LRU cache with a doubly linked list and dictionary.
- ☐ **930.** Implement a binary tree with pre-order, in-order, post-order and level-order traversals.
- ☐ **931.** Find the height and diameter of a binary tree.
- ☐ **932.** Check whether a binary tree is a valid binary search tree.
- ☐ **933.** Find the lowest common ancestor of two nodes in a BST.
- ☐ **934.** Serialize and deserialize a binary tree.
- ☐ **935.** Implement an AVL tree with rotations.
- ☐ **936.** Implement a union-find (disjoint set) with path compression.
- ☐ **937.** Implement a segment tree for range sum queries.
- ☐ **938.** Implement a Fenwick tree (binary indexed tree).
- ☐ **939.** Implement a skip list with search and insert.
- ☐ **940.** Implement a bloom filter with several hash functions.

## Graphs and Dynamic Programming

### STUDY THESE TOPICS FIRST

- Graph representations (adjacency list/matrix)
- BFS and DFS
- Topological sort and cycle detection
- Dijkstra, Bellman-Ford, Floyd-Warshall
- Kruskal and Prim (MST)
- Dynamic programming: memoization and tabulation
- Classic DP: knapsack, LCS, edit distance, coin change
- Backtracking

- ☐ **941.** Represent a graph using an adjacency list and an adjacency matrix.
- ☐ **942.** Implement BFS and DFS for a graph.
- ☐ **943.** Find the shortest path in an unweighted graph using BFS.
- ☐ **944.** Detect a cycle in an undirected graph and in a directed graph.
- ☐ **945.** Perform a topological sort of a directed acyclic graph.
- ☐ **946.** Find connected components in a graph.
- ☐ **947.** Implement Dijkstra's algorithm using a heap.
- ☐ **948.** Implement the Bellman-Ford algorithm to handle negative edges.
- ☐ **949.** Implement Floyd-Warshall for all pairs shortest paths.
- ☐ **950.** Implement Kruskal's and Prim's algorithms for a minimum spanning tree.
- ☐ **951.** Solve a maze given as a grid using BFS and print the path.
- ☐ **952.** Count the number of islands in a grid.
- ☐ **953.** Solve the 0/1 knapsack problem with dynamic programming.
- ☐ **954.** Find the longest common subsequence of two strings.
- ☐ **955.** Find the edit distance between two strings using dynamic programming.
- ☐ **956.** Solve the coin change problem for minimum coins and number of ways.
- ☐ **957.** Find the longest palindromic substring of a string.
- ☐ **958.** Solve the matrix chain multiplication problem.
- ☐ **959.** Solve the word break problem using dynamic programming.
- ☐ **960.** Solve a 9x9 Sudoku using backtracking.

## Concurrency, Networking and Performance

### STUDY THESE TOPICS FIRST

- threading, Lock and queue.Queue
- concurrent.futures: ThreadPoolExecutor and ProcessPoolExecutor
- multiprocessing and the GIL
- asyncio: coroutines, tasks, gather, Queue, Semaphore
- socket programming and HTTP basics
- Flask or FastAPI and Pydantic
- NumPy vectorization, numba/Cython basics
- Profiling and memory optimisation

- ☐ **961.** Create and start two threads that print numbers concurrently.
- ☐ **962.** Use a threading.Lock to prevent a race condition on a shared counter.
- ☐ **963.** Use queue.Queue to implement the producer-consumer pattern with threads.
- ☐ **964.** Use ThreadPoolExecutor to download several URLs in parallel.
- ☐ **965.** Use multiprocessing.Pool to compute squares of a large list in parallel.
- ☐ **966.** Compare threads and processes for a CPU-bound and an I/O-bound task.
- ☐ **967.** Explain and demonstrate the effect of the GIL on CPU-bound threads.
- ☐ **968.** Write an asyncio program that runs three coroutines concurrently.
- ☐ **969.** Use asyncio.gather and asyncio.wait\_for with a timeout.
- ☐ **970.** Use an asyncio.Queue for a producer-consumer pipeline.
- ☐ **971.** Fetch multiple web pages concurrently using aiohttp or httpx.
- ☐ **972.** Use asyncio.Semaphore to limit concurrent tasks.
- ☐ **973.** Write a TCP echo server and client using the socket module.
- ☐ **974.** Write a simple chat server that supports multiple clients using asyncio.
- ☐ **975.** Create a simple REST API using Flask or FastAPI with GET and POST endpoints.
- ☐ **976.** Add input validation to an API endpoint using Pydantic.
- ☐ **977.** Optimize a slow function using NumPy vectorization and measure the speedup.
- ☐ **978.** Use functools.lru\_cache and then Cython or numba to speed up a numeric loop.
- ☐ **979.** Profile a program and reduce memory use by using generators and \_\_slots\_\_.
- ☐ **980.** Write a program using concurrent.futures that processes files in parallel and reports progress.

## Testing, Packaging and Capstone Projects

### STUDY THESE TOPICS FIRST

- unittest and pytest (fixtures, parametrize)
- unittest.mock
- coverage.py and hypothesis
- pyproject.toml, building wheels and entry points
- pre-commit, Docker basics
- SQLite, Flask, BeautifulSoup, requests
- pandas, matplotlib and scikit-learn basics
- Project structure and Git

- ☐ **981.** Write unit tests for a calculator using unittest.
- ☐ **982.** Write the same tests using pytest with parametrize.
- ☐ **983.** Use fixtures in pytest to set up and tear down test data.
- ☐ **984.** Use unittest.mock to replace a network call in a test.
- ☐ **985.** Measure test coverage using coverage.py and improve it.
- ☐ **986.** Write a property-based test using the hypothesis library.
- ☐ **987.** Package a small project with pyproject.toml and build a wheel.
- ☐ **988.** Create a command-line tool installable with an entry point.
- ☐ **989.** Add type checking and linting to a pre-commit configuration.
- ☐ **990.** Create a Dockerfile to run a Python script and build the image.
- ☐ **991.** Build a command-line to-do app that stores tasks in a JSON file.
- ☐ **992.** Build a contact book application using SQLite and a CLI menu.
- ☐ **993.** Build a web scraper that extracts headlines from a news page using BeautifulSoup.
- ☐ **994.** Build a weather app that calls a public API and prints a forecast.
- ☐ **995.** Build an expense tracker with CSV storage and a monthly summary report.
- ☐ **996.** Build a URL shortener web service with Flask and a database.
- ☐ **997.** Build a simple Markdown to HTML converter.
- ☐ **998.** Build a tiny interpreter for arithmetic expressions with variables.
- ☐ **999.** Analyze a CSV dataset with pandas and plot three charts with matplotlib.
- ☐ **1000.** Train a simple machine learning classifier with scikit-learn and report its accuracy.